

PROJECT VARDAN

Engineering Challenges, Data Bottlenecks & Architectural Solutions

1. Synthetic Climate Data & 100-Year Longitudinal Sparsity

The Problem: Real-world observational degradation data spanning 100 years under severe future climate stress (temperature anomalies up to +4.0°C, elevated storm events, sea-level saline intrusion) does not exist in empirical historical datasets. Furthermore, novel sustainable construction materials (e.g., blast furnace slag cement, alkali-activated geopolymers) were developed only recently.

Engineering Solution: Built **physics-informed degradation generation functions** anchored to empirical baseline embodied carbon from the certified Circular Ecology ICE V5 dataset. Applied strict boundary constraints (non-negativity $\Delta GWP \geq 0$ and thermodynamic conservation) to prevent runaway overfitting or unrealistic negative emissions.

2. Legacy Dataset Heterogeneity & Multilingual Encoding Collisions

The Problem: Raw European Environmental Product Declaration (EPD) databases (such as German OEKOBAU.DAT / Traditional_Construction_Materials.csv) contained ISO-8859-1 (Latin-1) encoding collisions, semicolon delimiters, and German environmental nomenclature (e.g., 'Schuettdichte (kg/m3)', 'Name (de)'), which caused UnicodeDecodeError and unit mismatches during standard data ingestion.

Engineering Solution: Standardized an automated multi-encoding normalization pipeline that parsed, translated, and structured 259 verified construction materials into a clean, UTF-8 certified database (data/ICE_V5_Cleaned_Materials.csv) with uniform kg CO2e / kg metric units.

3. Hybrid Architecture (Deterministic Physics + Stochastic ML)

The Problem: Under ISO 21930 / EN 15978 standards, Stage A1–A4 supply chain accounting must follow exact, zero-variance mathematical formulas, whereas 100-year operational resilience (B1–B7) requires statistical machine learning. Combining these paradigms in one unified REST API risked rounding drift and latency.

Engineering Solution: Built decoupled service layers: material_service.py (O(1) in-memory cache), transport_service.py (analytical mass-penalty engine, RMSE = 0.0), and prediction_service.py (lazy-loaded Scikit-Learn Random Forest Regressor), safeguarded by Pydantic request validations and zero-division protection.

4. Model Serialization & Cloud Deployment Drift (Render)

The Problem: InconsistentVersionWarning and unpickling errors arose when loading joblib models across differing Scikit-Learn versions (1.6.1 vs 1.9.0), while static relative paths failed across differing Linux cloud container working directories on Render.

Engineering Solution: Implemented dynamic 4-tier absolute path resolution in prediction_service.py, selective warning suppression, and an infrastructure-as-code render.yaml blueprint locking Python 3.11.9 and Node 20.12.2 environments.

5. Multi-Stage Lifecycle UI/UX & Dual Metric Accounting

The Problem: Displaying 4 discrete Life Cycle Assessment stages simultaneously can overwhelm end-users and cause confusion between total journey emissions (kg CO2) and normalized material footprint (kg CO2e / kg material).

Engineering Solution: Created an interactive 4-stage comparison dashboard in React with step-by-step verification cards and dual-metric clarity.

6. Challenges & Solutions Matrix

Challenge Area	Primary Constraint	Applied Resolution	Outcome Metric
Synthetic Data	No 100-yr empirical future data	Physics-informed IPCC scenario bounds	$R^2 = 0.9942$, MAE = ± 0.0084
Legacy Encoding	Latin-1 / German delimiter collisions	Cleaned 259-material UTF-8 registry	100% clean ingestion
Hybrid Pipeline	Merging physics with ML models	Decoupled FastAPI service architecture	RMSE = 0.00 (A4), 99.15% (ML)
Deployment Drift	Version mismatch & path errors	Hierarchical path resolver + render.yaml	Zero downtime cloud build
UI/UX Clarity	Conflating trip CO2 vs material CO2	4-Stage lifecycle visual breakdown	Instant interactive feedback